Lecture 08 · Data Structure(자료 구조)

강의 · Notion 아카이브

list comprehension(리스트 컴프리헨션)

- **list(리스트)**를 생성하기 위한 간결하고 표현력이 높은 문법
- 표현식: 리스트에 담길 항목을 생성
- for 변수 in 반복가능한**Object(객체)**: 반복을 수행할 변수와 반복할 대상
- if 조건문: 특정 조건을 만족하는 항목만 리스트에 포함
  - 기본 형태: [표현식 for 변수 in 반복가능한객체]
    if 사용: [표현식 for 변수 in 반복가능한객체 if 조건문]
    if-else 사용: [표현식 if 조건문 else 대체값 for 변수 in 반복가능한객체]
    다중 for: [표현식 for 변수1 in 반복가능한객체1 for 변수2 in 반복가능한객체2]

```
x=[]
for i in range(10):
    x.append(i)

y=[i for i in range(10)]
```

  - ⇒ x = y = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
- 필터링 [if ~]

```
y = [i for i in range(10) if i % 2 == 0]
```

  - ⇒ y = [0, 2, 4, 6, 8] # 짝수만 저장
  - if ~ else ~

```
y = [i if i%2==0 else 99 for i in range(10)]
```

  - ⇒ y= [0, 99, 2, 99, 4, 99, 6, 99, 8, 99] # else로 홀수 → 99
- nested loop (반복문 안에 또다른 반복문이 있는 것을 list에서 표현)

```
r = []
for i in "Hello":
for j in "World":
                    r.append(i + j)
# 를 아래와 같이 표현할 수 있음
```

```
w1="Hello"
w2="World"
r=[i+j for i in w1 for j in w2]
```

- ['HW', 'Ho', 'Hr', 'Hl', 'Hd', 'eW', 'eo', 'er', 'el', 'ed', 'lW', 'lo', 'lr', 'll', 'ld', 'lW', 'lo', 'lr', 'll', 'ld', 'oW', 'oo', 'or', 'ol', 'od']
- 필터링

```
c1=["A","B","C"]
c2=["D","E","A"]
r=[i+j for i in c1 for j in c2 if not (i==j)] # i == j 필터링
```

- ['AD', 'AE', 'BD', 'BE', 'BA', 'CD', 'CE', 'CA']

2단부터 9단까지 구구단 생성

```
gugudan = [(a, b, a*b) for a in range(2, 10) for b in range(1, 10)]
for item in gugudan:
    print(f"{item[0]} x {item[1]} = {item[2]}")
```

default arguments (디폴트 인수)

- **Function(함수)**에서 입력 값에 디폴트 값을 정할 수 있음

```
def func_sum(a,b,c=10,d=20): # 디폴트 값은 다른 변수들 뒤에 위치해야 함.
sum=a+b+c+d
return(sum)
print(func_sum(10,20)) # 60 출력
print(func_sum(10,10,10,10)) # 40 출력
```

- ! 디폴트 값은 다른 변수들 뒤에 위치해야 함.

variable length arguments (가변 인수)

- *args ⇒ 함수에 임의의 개수의 위치 인자를 전달할 수 있게 해줌
- 인자들은 함수 내부에서 **tuple(튜플)**로 처리

```
def print_args(*args):
for arg in args:
        print(arg)
print_args(1, 2, 3 ,4) # 1. 2. 3. 4. 출력
```

- keyword variable length arguments (키워드 가변 인수)
- **kwargs ⇒ 함수에 임의의 개수의 키워드 인자를 전달할 수 있게 해줌
- 인자들은 함수 내부에서 **dictionary(딕셔너리)**로 처리

```
def print_kwargs(**kwargs):
for key, value in kwargs.items():
        print(f"{key} = {value}")
```

```
print_kwargs(a=1, b=2, c=3)
# a = 1 b = 2 c = 3 출력
```

Stack(스택)

- 나중에 들어온 것이 먼저 나간다(LIFO, Last-In First-Out)

```
stack = []
stack.append('a')
stack.append('b')
stack.append('c')
print('Initial stack:', stack)
print('\nElements popped from stack:')
print(stack.pop())
print(stack.pop())
print(stack.pop())
print('\nStack after elements are popped:', stack)
```

```
Initial stack: ['a', 'b', 'c']

Elements popped from stack:
c
b
a

Stack after elements are popped: []
```

- 출력, 즉 a, b, c 로 입력 ⇒ c, b, a 순서로 출력

queue (큐)

- 먼저 들어온 것이 먼저 나간다(FIFO, First-In First-Out)

```
queue = []
queue.append('a')
queue.append('b')
queue.append('c')
print('Initial queue:', queue)

print('\nElements dequeued from queue:')
print(queue.pop(0))
print(queue.pop(0))
print(queue.pop(0))
```

```
print('\nQueue after elements are dequeued:', queue)
```

```
Initial queue: ['a', 'b', 'c']

Elements dequeued from queue:
a
b
c

Queue after elements are dequeued: []
```

- a, b, c 입력 ⇒ a, b, c 순서 출력

set(집합)

- 순서없이 저장, 데이터 중복을 불허

```
s = set ([1,2,3,4,5,1,2,3])
print(s)
ss = {10,20,30,40,50,10,20,30}
print(ss)
```

```
{1, 2, 3, 4, 5}
{50, 20, 40, 10, 30}
```

- Set example 1

```
my_set = set()

my_set.add(1) ## add 함수
my_set.add(2)
my_set.add(3)

print(my_set) # Output: {1, 2, 3}

my_set.update([1,3,4,5,6,7]) ## 업데이트 함수

print(my_set) # Output: {1, 2, 3, 4, 5, 6, 7}

my_set2 = {10, 20, 30}

my_set2.add(40)
```

```
my_set2.add(10)
my_set2.add(50)

my_set2.remove(20)

print(my_set2) # Output: {40, 10, 50, 30}
```

  - **.add** **.update** 사용할 수 있음.

- Set example 2

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}

# 합집합(Union)
print(set1.union(set2)) # Output: {1, 2, 3, 4}
print(set1 | set2)      # Output: {1, 2, 3, 4}

# 교집합(Intersection)
print(set1.intersection(set2)) # Output: {2, 3}
print(set1 & set2)             # Output: {2, 3}

# 차집합(Difference)
print(set1.difference(set2)) # Output: {1}
print(set1 - set2)           # Output: {1}
```

  - **.union** ⇒ set1 | set2
  - **.intersection** ⇒ set1 & set2
  - **.difference** ⇒ set1 - set2

collections module(모듈)

- collections : 파이썬 내장 모듈
- deque : 양쪽 끝에서 삽입과 삭제 가능, 스택 큐 모두 지원

```
from collections import deque

queue = deque([1, 2, 3])
print(queue)
queue.append(4)
print(queue)
queue.append(5)
print(queue)
```

```
queue.popleft() ## => 앞에서부터 없앰
print(queue)
queue.pop()   ## => 뒤에서부터 없앰
print(queue)
```

```
deque([1, 2, 3])
deque([1, 2, 3, 4])
deque([1, 2, 3, 4, 5])
deque([2, 3, 4, 5])
deque([2, 3, 4])
```

- OrderedDict : 입력된 순서로 key-value 저장, key 또는 value로 데이터 **Sort(정렬)** 가능

```
from collections import OrderedDict


d = OrderedDict()
d['b'] = 100
d['a'] = 700
d['d'] = 300
d['c'] = 500
d['e'] = 200

for key, value in d.items():
    print(key, value)
```

```
b 100
a 700
d 300
c 500
e 200
```

- example OrderedDict

```
from collections import OrderedDict


def sort_by_key(t):
      return t[0] #t[0]-key, t[1]-value


d = dict()
d['b'] = 100
d['a'] = 700
d['d'] = 300
d['c'] = 500
```

```
d['e'] = 200


for key,value in OrderedDict(sorted(d.items(), key=sort_by_key)).items():
    # sorted(d.items(), key=sort_by_key) => key의 순서(a,b,c,d,e)로 정렬
    print(key,value)
```

```
a 700
b 100
c 500
d 300
e 200
```

- Counter : 데이터 값의 개수를 딕셔너리 형태로 변환

```
from collections import Counter


fruits = ['apple', 'banana', 'cherry', 'apple', 'cherry', 'apple', 'banana']


fruit_count = Counter(fruits)


print(fruit_count)
```

```
Counter({'apple': 3, 'banana': 2, 'cherry': 2})
```

← 프로그래밍과 문제해결 · CSED101 목차로